# VCS$^3$ 3D Kit

# Getting started guide

## Table of Contents



| Version | Notes | Initials | Date |
|---|---|---|---|
| **0.1** | Initial Version | CH | 21st July 2025 |
| | | | |
| | | | |

# Kit contents

The VCS3 is a compact single unit with multiple I/Os already connected.

It is comprised of the unit itself and a 5V USB-C power supply (UK customers only) – Not pictured.

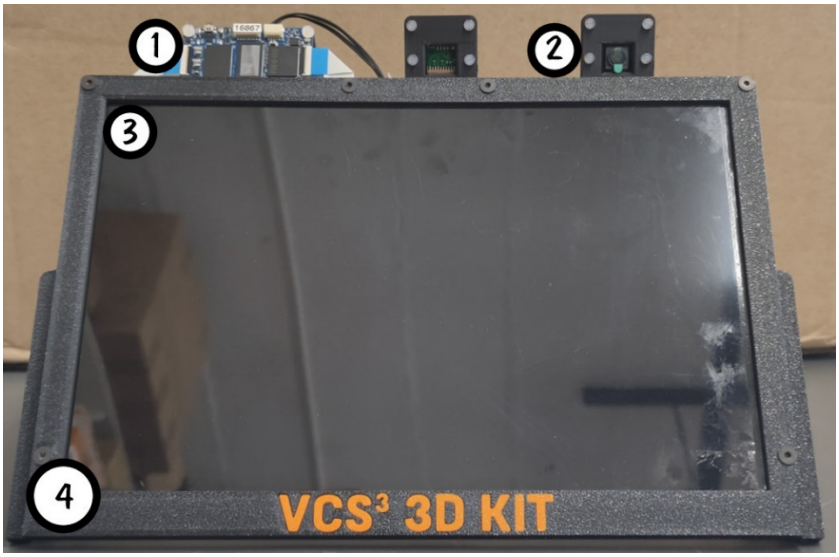


*Figure 1 VCS³ 3D kit - Front View*

1. The VCS³ FPGA module. This has 4x 22 pin MIPI connectors for CSI and DSI. For this demo we are using 2x for cameras and 1x for the DSI screen.
2. Camera. Pi camera V2. https://www.raspberrypi.com/products/camera-module-v2/
3. DSI screen. This is the CLAA101FP05 B101UAN01.7 as used in other AMD FPGA demos. It's a 10.1" 1920x1200 screen.
4. 3D printed holder for the system.

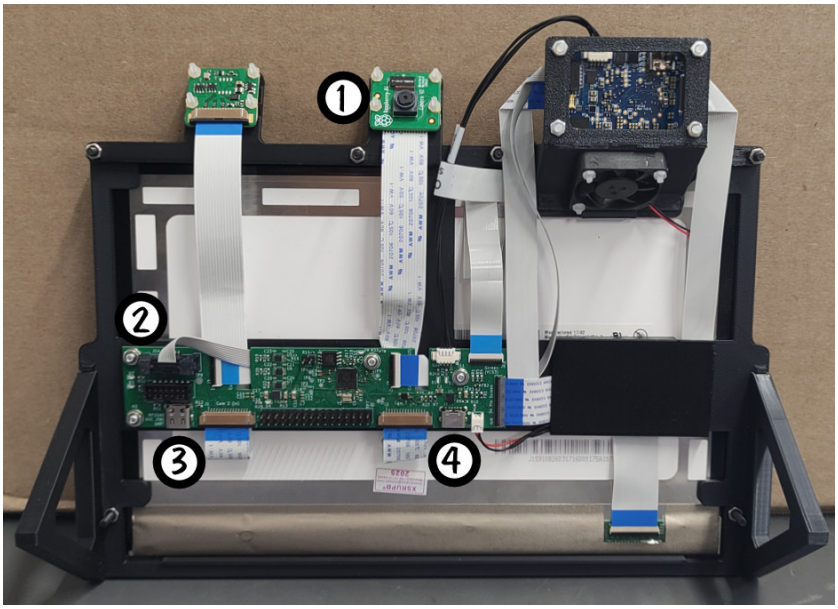


*Figure 2 VCS³ 3D Kit rear view*

1. Camera.  Pi camera V2. https://www.raspberrypi.com/products/camera-module-v2/
2. VCS3-RP2040 Converter card.  More info below.
3. USB-C for XVC and UART.
4. USB-C (5V) for powering the VCS3, fan and screen back light.

The VCS3-RP2040 is a Pi RP2040 powered converter card for MIPI interface adaption, UART communication and XVC JTAG.

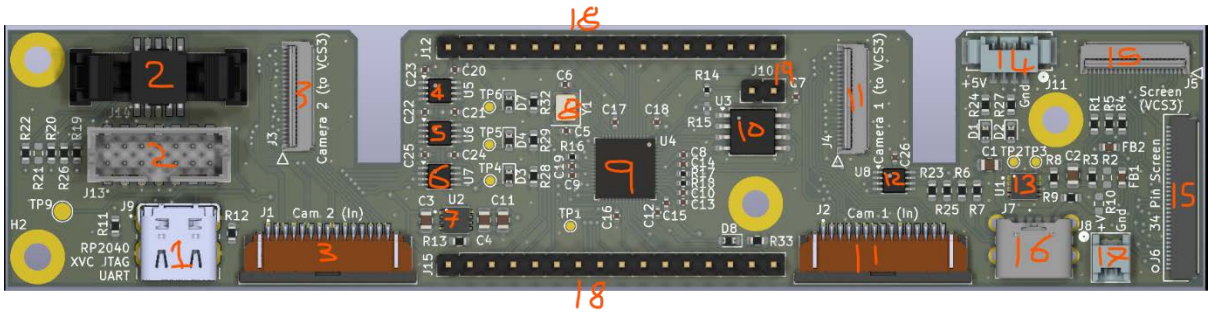


*Figure 3 VCS3-RP2040 IO card*

1. USB-C for Data comms. This is main USB connection for the Pi RP2040 chip. When programmed this is where the JTAG XVC and VCS3 UART come out. It also powers the left hand side of the board, so the RP2040 and the level shifters.
2. JTAG headers.
3. Camera converter between the 15 pin MIPI (Used on the Pi Camera V2) and the 22 pin MIPI (Used on the VCS3)
4. This is a level shifter for JTAG.
5. This is a level shifter for JTAG.
6. This is a level shifter for JTAG.
7. A LDO DCDC to convert 5V (from the USB connector) to 3.3V for the Pi RP2040.
8. The crystal for the processor.
9. The processor (RP2040).
10. FLASH for the processor.
11. Camera converter between the 15 pin MIPI (Used on the Pi Camera V2) and the 22 pin MIPI (Used on the VCS3)
12. Level shifter used as a buffer to remove the “powered via UART” issue.
13. A LDO DCDC to convert 5V (from the USB connector) to 3.3V
14. DC Power and UART to the VCS3
15. This is a screen converter between the 34 pin MIPI and the 22 pin MIPI. This also has the extra 3.3V on it from (13) to power the back light.
16. USB-C for power only. Powers the VCS3, 3.3V LDO for the screen back light and the buffer to sort out the UART TX signal.
17. Fan connector. Can be powered from 5 or 3.3V. Now by default, powered from 5V.
18. Spare GPIO pins from the RP2040.
19. Header to put the RP2040 into programming mode.

# Hardware Setup

As previously mentioned, all the I/Os should be connected for you when you get it out of the box.  All you should need to do is connect one USB-C power supply to the right-hand USB-C connector (J7) for power.

The other USB-C (J9) connector is for data communication (UART and XVC.  More features may be added later).

# The VCS$^3$-RP2040

## About the VCS$^3$-RP2040

The I/O card the VCS$^3$-RP2040 is used for XVC JTAG as well as UART communication to the VCS$^3$ module.

It will come programmed with the required code to run the UART and XVC JTAG.  If required, a link to our fork of the master GitHub repository can be provided for re-programming.

It is firmware and software is based on the GitHub project xvc-pico (https://github.com/kholia/xvc-pico).  This base project has a couple of issues we have rectified, so please ask if you need to re-program it.

The VCS$^3$-RP2040 has conversion between the standard 15 pin MIPI connector from the Raspberry Pi V2 cameras to the 22 pin MIPI connector used by the VCS$^3$.

It also converts the 22 pin MIPI on from the VCS$^3$ to the 34 pin connector used by the 10.1” screen.  Extra power is supplied to power the backlight to assist the VCS3.

# Using the VCS3-RP2040 as a XVC JTAG pod.

## Setting up the software under Linux

(Taken from https://github.com/kholia/xvc-pico"

**Building pico-xvc (for Linux users)**

Install dependencies:

```
sudo apt install cmake gcc-arm-none-eabi libnewlib-arm-none-eabi \
  libstdc++-arm-none-eabi-newlib git libusb-1.0-0-dev build-essential \
  make g++ gcc
mkdir ~/repos
cd ~/repos

git clone https://github.com/raspberrypi/pico-sdk.git
cd pico-sdk; git submodule update --init

cd ~/repos
git clone https://github.com/kholia/xvc-pico.git
Build the host-side daemon:
cd ~/repos/xvc-pico/daemon
cmake .
make
```

You only need to this once.

Now, go to the right directory and run the daemon to enable the host side XVC software:

```
cd ~/repos/xvc-pico/daemon
```

Connect the VCS[3]-RP2040 to your computer if not already, and then run

sudo ./xvcd-pico

After entering your password you should see the following:

```
christopherh@Latitude-7390:~/repos/xvc-pico/daemon$ sudo ./xvcd-pico
[sudo] password for christopherh:
NB: ep_size => 64
XVCPI is listening now with BUFFER_SIZE => 524288!
```

*Figure 4 output of the command under LINUX*

Leave this running and then launch the AMD tool you require. We are going to show you the procedure to use vivado hardware manager.

## Under Windows

Grab **xvcd-pico.exe** from the builds folder of this repository itself. (https://github.com/kholia/xvc-pico)

You need to install the libusbK driver with Zadig (https://zadig.akeo.ie/).

OR

You can install the WinLibUSB driver from USB Drive Tool Application (https://visualgdb.com/UsbDriverTool/).

Credit goes to https://github.com/benitoss/ for these instructions.

**Usage**

On the host computer with the Raspberry Pi Pico connected, run the XVC Daemon Server (**xvcd-pico.exe**) from where you downloaded it.

You should see a box like this:

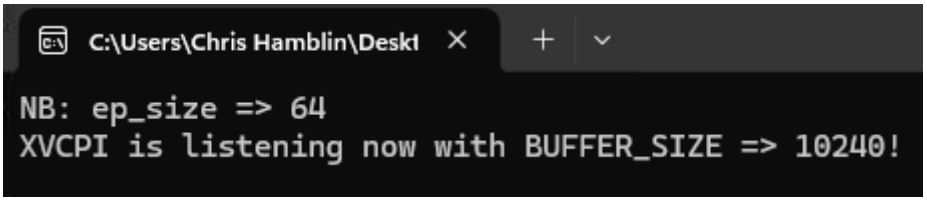


*Figure 5 - output of the .exe under windows*

Leave this open while you are using the XVC.

## Using a XVC in Vivado.

Open Vivado and select “Open Hardware Manager”:

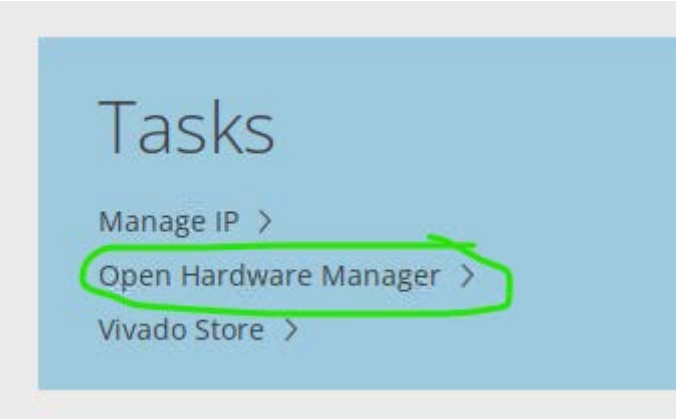


*Figure 6 - Vivado Hardware manager*

Start the Open New Target wizard: “Tools -> Open new Target”
Press “Next” on the first screen.
On the next screen, ensure that Local Server is selected. Press next.

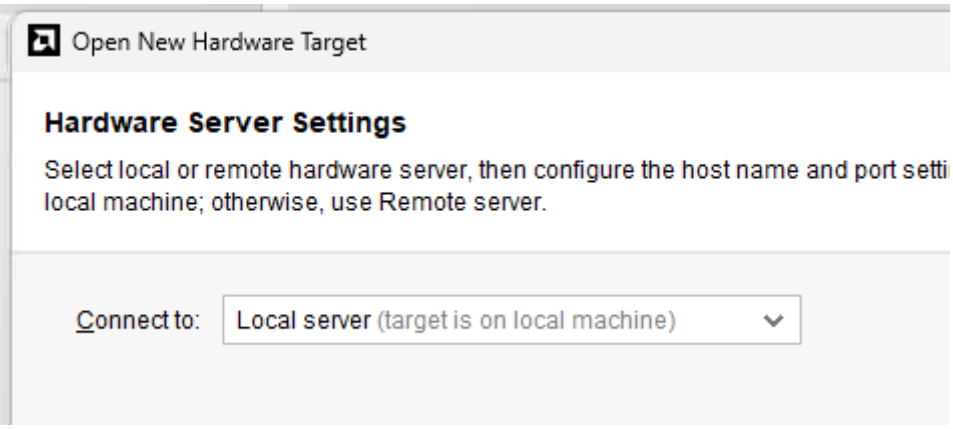


*Figure 7 - Open Hardware Wizard Local Server*

The next screen will look blank, this is OK.
Press the Add Xilinx Virtual Cable (XVC) button.

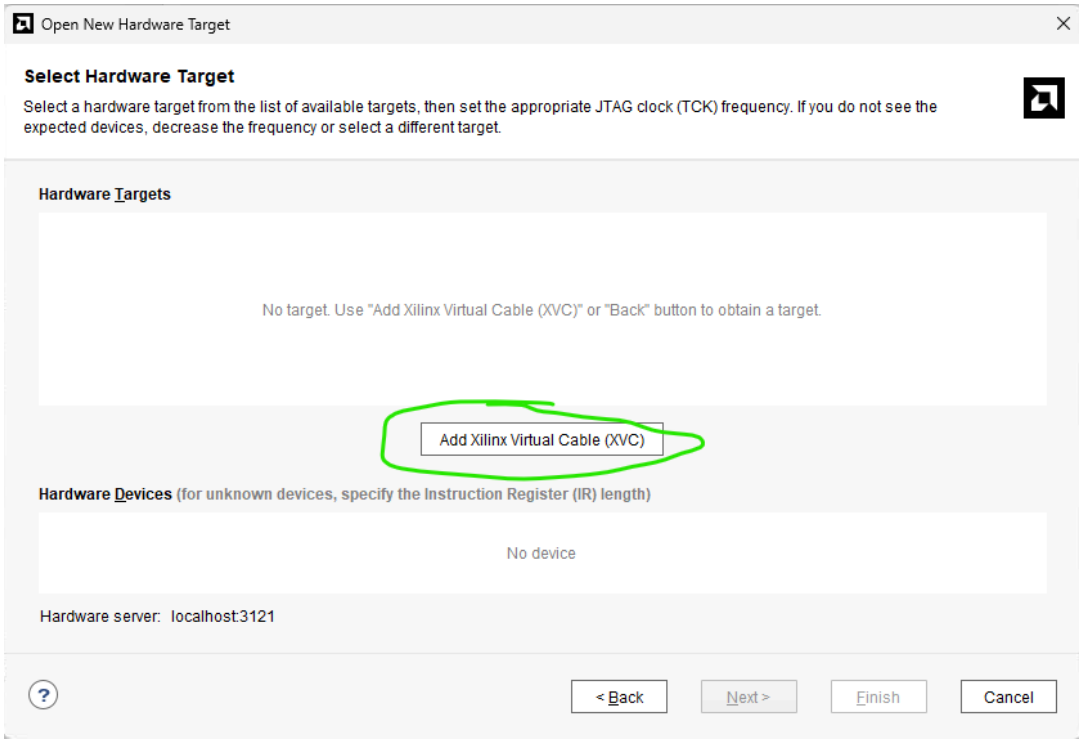


*Figure 8 - Add Xilinx Virtual Cable*

In the box that pops up, set the host name to “localhost.” Leave the port as 2542.

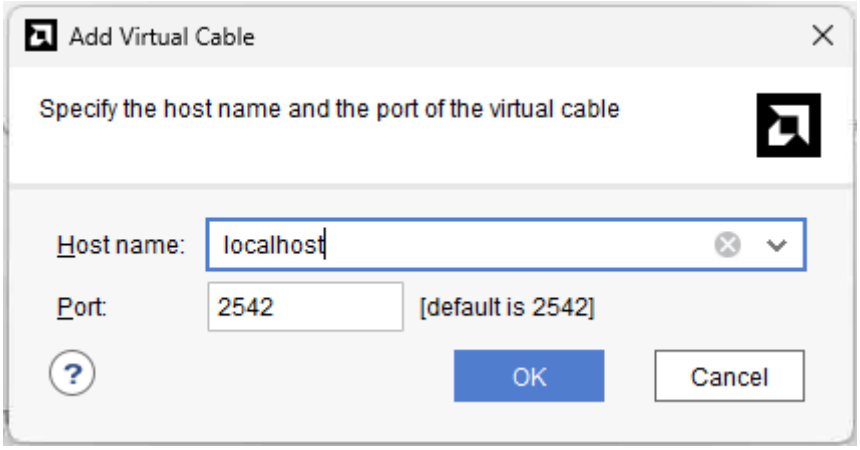


*Figure 9 - Virtual Cable Settings*

Press OK.

The wizard then connects to the XVC cable and checks what is connected. You will see a clock frequency of 1000000.

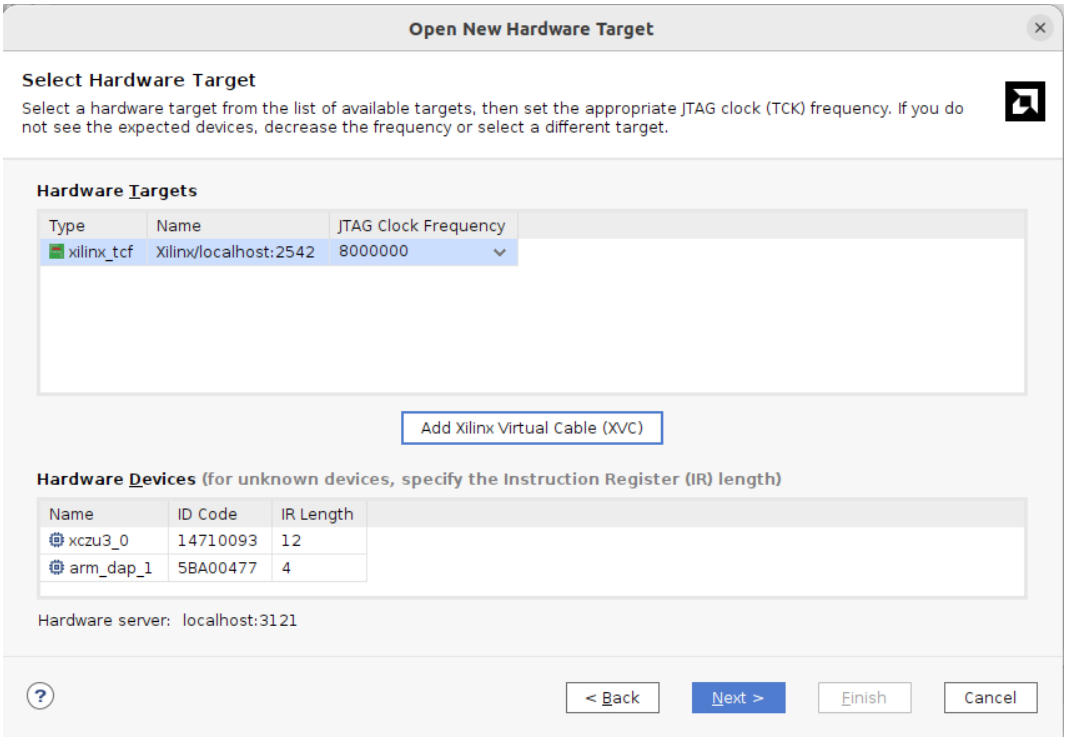


*Figure 10 - Virtual Cable: Good Result*

It should show like above, where there is a hardware target listed, and two hardware devices show, the xczu3 and the arm core.

Press next, then finish.

It will then connect and refresh the devices. You can now use it as normal.

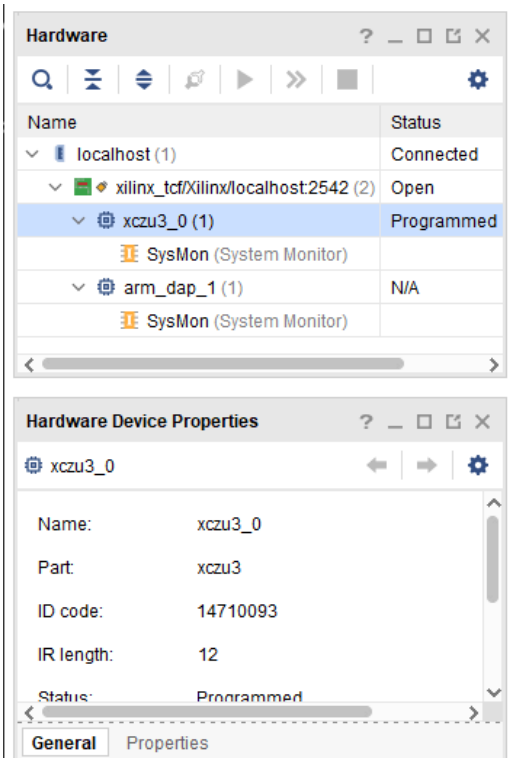


*Figure 11 - Vivado Connected Devices*

# Using the VCS$^3$-RP2040 as a UART connection

## Under LINUX

Plug the lefthand USB-C connector into your computer.  It runs at less than USB 2 speeds so any USB-A – USB-C cable that supports data is fine, as well as any USB-C – USB-C cable.

Once connected, you should now have a "/dev/ttyACM(n)" device which can be connected by the UART communicator of your choice (screen, minicom, putty etc. etc.).

The connection baud rate is 115200.

If you "lsusb" you should see a "Raspberry Pi Pico" listed.

## Under Windows

Plug the lefthand USB-C connector into your computer.  It runs at less than USB 2 speeds so any USB-A – USB-C cable that supports data is fine, as well as any USB-C – USB-C cable.

Once connected, you should now have a new comport device listed in the hardware manager which can be connected by the UART communicatior of your choice (teraterm, putty etc. etc.).

The connection baud rate is 115200.

# Using the built in Demo

To use the built in demo, power up the system using the right hand USB-C connector.  This should then boot a modified version of PetaLinux and launch the demo.

You will then see live images from both cameras on screen, with the large image swapping places with the small image, jumping from front to rear cameras.

# About the VCS3

## Description

***The VCS3 is 'probably'* the smallest single-board computer based on an AMD Zynq MPSoC.**

An ultra-compact, low-power, vision, control, and sensors solution for precision robotics.

The $VCS^3$ is a small single-board computer with an AMD© ZYNQ™ device with integrated ARM CPUs and FPGA fabric. Measuring just 30mm x 50mm, this tiny workhorse can be placed almost anywhere, opening the benefits of FPGAs to many more applications.

The $VCS^3$ utilizes an AMD UltraScale+ MPSoC coupled with high-speed LP-DDR4 memory to produce a highly compact evaluation platform. Together with four digital camera interfaces, a 9-axis IMU, and a CAN-bus interface, this platform is ideally suited for autonomous machines, cameras, or automation.

Device booting can be from SPI ROMs, eMMC flash or for development, JTAG.

Numerous onboard power rails are generated from a single external 5V supply, either from a dedicated connector or via a USB3 Type-C interface.

Several LEDs indicate board functionality, and eight connectors allow access to the various interfaces.

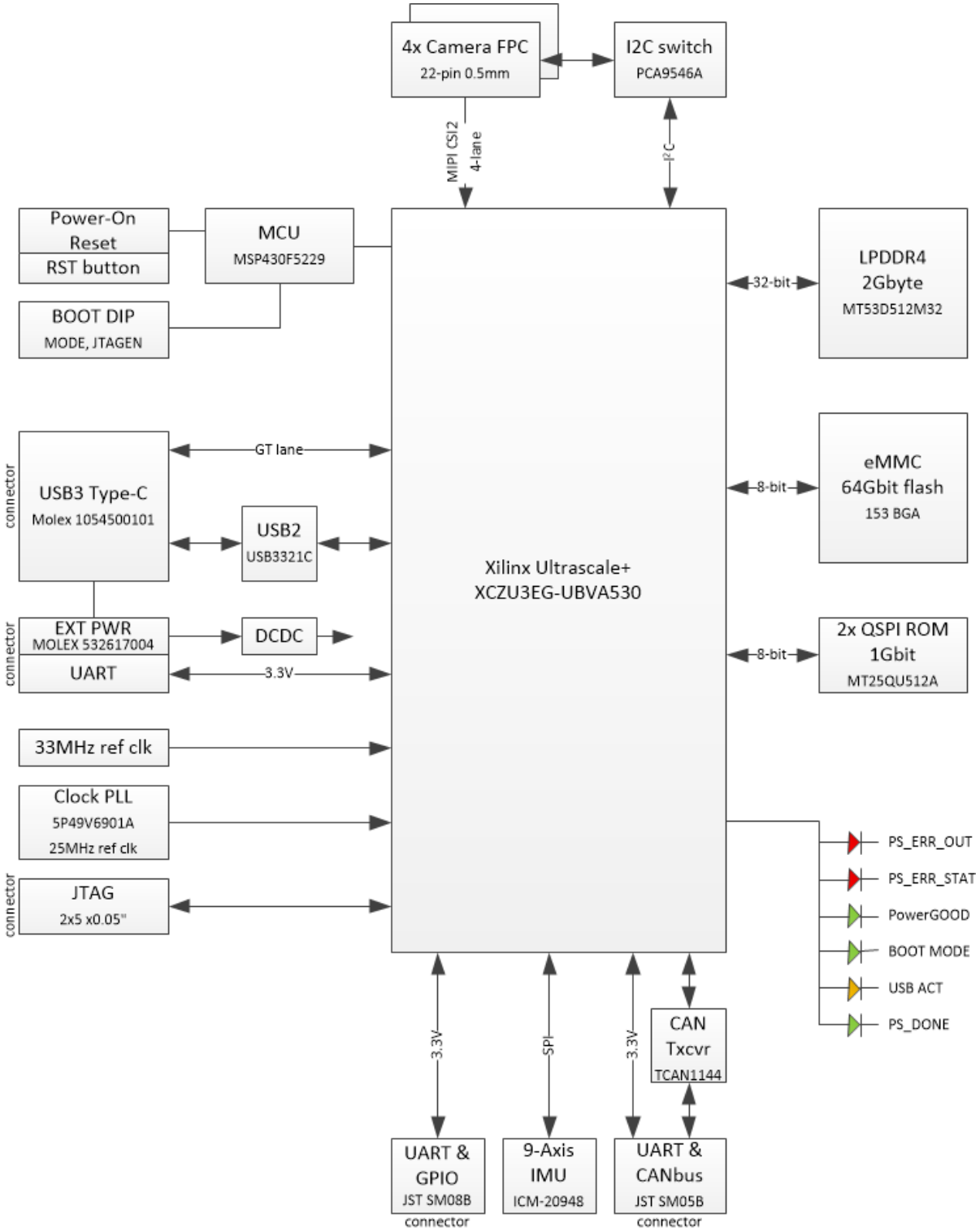


*Figure 12 Block Diagram of the VCS[3]*

## Parts of the VCS$^3$

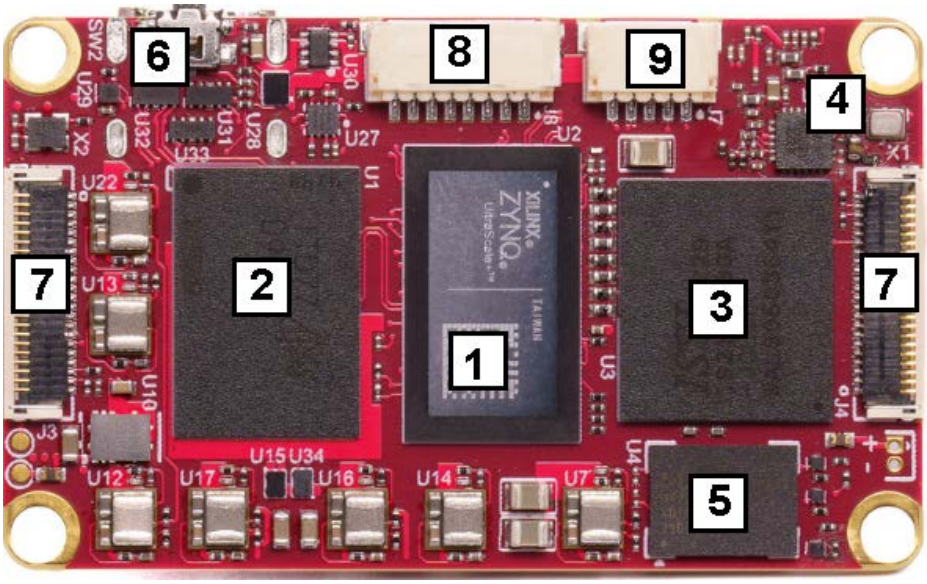


*Figure 13 - VCS$^3$ Top*

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| **1** | **AMD Zynq MPSoC** | **2** | **LP-DDR4** | **3** | **eMMC** | **4** | **9-axis IMU** | **5** | **SPI ROM** |
| **6** | **Reset Switch** | **7** | **MIPI Camera Connectors** | **8** | **GPIOs** | **9** | **CAN BUS and UART** | **10** | **JTAG header** |
| **11** | **BOOT Mode Switch** | **12** | **CAN BUS Interface** | **13** | **Power and UART** | **14** | **Microcontroller** | **15** | **USB3 Type C** |

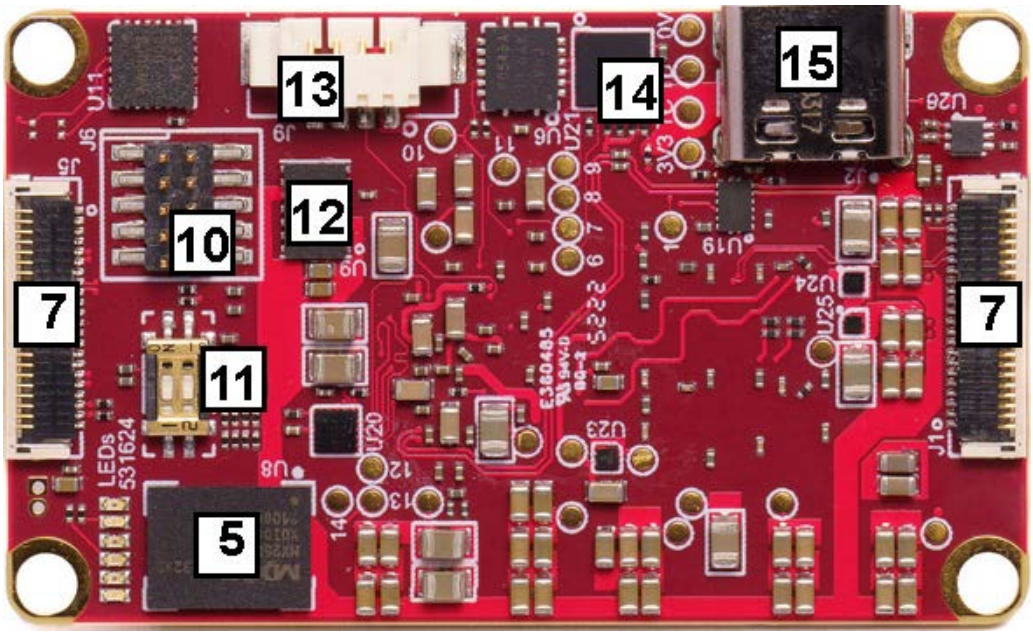


*Figure 14 - VCS$^3$ Bottom*

## Boot Options

There are three boot options at present for the VCS[3]. These are controlled from SW1 (Item 11 above).

| Switch 1 | Switch 2 | |
|---|---|---|
| **OFF** | **OFF** | **JTAG** |
| **ON** | **OFF** | **eMMC** |
| **OFF** | **ON** | **QSPI ROM** |
| **ON** | **ON** | |